CROPCYCLE

AI Backend Integration Contract

Proposed schemas, current backend capabilities, API boundaries, validation, offline behavior, and decisions to confirm before connecting an AI provider.

Prepared for the CropCycle AI integration team · 10 October 2026

This document consolidates the integration response. It distinguishes implemented CropCycle behavior from proposed interfaces so both teams can agree on the contract before implementation. Example IDs and values below are illustrative, not production records.

1. AI input schema

PROPOSED CONTRACT CropCycle's backend should assemble and validate the request from its database, then call the AI service server-to-server. Do not send passwords, session tokens, farmer names, phone numbers, or other direct identifiers.


```
{
  "schemaVersion": "1.0",
  "requestId": "req_example_001",
  "farmerId": "AGR-F-000001",
  "farmId": "AGR-FM-000128",
  "plotId": "AGR-PL-000401",
  "seasonId": "clxseasonexample",
  "season": {
    "name": "2026 wet season",
    "status": "PLANNING",
    "farmingMethod": "MIXED_FARMING",
    "location": {
      "region": "Kaduna North, Kaduna",
      "latitude": null,
      "longitude": null
    }
  },
  "cropRequirements": {
    "intendedCrops": ["crop-id-from-catalog"],
    "availableCrops": ["crop-id-from-catalog"],
    "marketableCrops": ["crop-id-from-catalog"],
    "preferredFarmingMethod": "MIXED_FARMING",
    "otherRequirements": null
  },
  "previousSeasons": [{
    "seasonId": "clxpreviousseason",
    "seasonName": "2025 dry season",
    "farmingMethod": "MONO_CROPPING",
    "cropsPlanted": ["crop-id-from-catalog"],
    "recommendations": [{
      "recommendationId": "clxrecommendation",
      "version": 1,
      "createdAt": "2025-04-12T09:30:00.000Z",
      "status": "SHARED",
      "recommendedCropIds": ["crop-id-from-catalog"],
      "outcome": {
        "actualCropIds": ["crop-id-from-catalog"],
        "yieldQuantity": null,
        "yieldUnit": null,
        "farmerRating": null,
        "feedback": null
      }
    }]
  }],
  "soilReadings": [{
    "readingId": "clxreading",
    "recordedAt": "2026-04-12T09:30:00.000Z",
    "nitrogen": { "value": 45, "unit": "ppm" },
    "phosphorus": { "value": 18, "unit": "ppm" },
    "potassium": { "value": 110, "unit": "ppm" },
    "soilTemperature": { "value": 27.4, "unit": "°C" },
    "humidity": { "value": 62, "unit": "%" }
  }],
  "approvedKnowledge": [{
    "knowledgeId": "clxknowledge",
    "key": "crop-region-key",
    "version": 2,
    "title": "Example approved rule",
    "sourceCitation": "Source citation",
    "effectiveFrom": "2026-01-01T00:00:00.000Z",
    "effectiveTo": null,
    "body": {}
  }]
}
```

Farmer and farm codes already exist. Season and reading IDs are internal IDs. The current database does not yet have a canonical Crop Scientist crop catalog, so the crop IDs shown here are placeholders and must not be used as real IDs.

The current recommendation endpoint selects only the latest soil reading for the current season and passes it to the rule engine as a single soil object with `recordedAt`. A dated readings array is a proposed AI contract, not the current endpoint payload.

2. Intended crops and canonical crop IDs

NOT IMPLEMENTED YET Intended, available, marketable, previous, and actual crops are currently stored mostly as text strings. There is no shared Crop table or crop-catalog endpoint.

Proposed next step: create a Crop Scientist-managed catalog with immutable crop IDs, display names, and optional aliases. Reuse those IDs in crop requirements, crop history, knowledge rules, AI input/output, recommendations, and outcomes. Until that exists, current APIs can pass crop names as text; they should not fabricate IDs or assume differently entered names are equivalent.

3. Previous seasons and recommendations endpoint

PROPOSED ENDPOINT There is no focused farm-history endpoint today.

Method and path	Purpose
`GET /api/farms/{farmCode}/history?plotId={plotCode}`	Return authorized seasons, crops by record type (PREVIOUS, INTENDED, ACTUAL), prior recommendations, outcomes, and feedback. Support pagination, explicit ordering, and ownership/membership authorization.

`GET /api/workspace` currently returns a dashboard-shaped, access-filtered payload with bounded recent recommendations and outcomes. It is not a definitive or unbounded history API and should not be the AI history source.

4. Crop Scientist rules and retrieval

Rules live in PostgreSQL in the `KnowledgeRule` model. Each record contains a version, JSON body, source citation, status, author/approver, approval time, and effective dates.

Route	Current behavior
`GET /api/knowledge`	Authenticated Crop Scientists/Admins can view all statuses; other users receive currently effective approved rules.
`POST /api/knowledge`	Crop Scientists/Admins create drafts with a structured body and source citation.
`PATCH /api/knowledge/{knowledgeId}`	Crop Scientists/Admins update drafts, approve drafts, or retire approved rules.

The recommendation backend uses currently effective approved rules and saves references to the rules it used. The AI must never receive drafts or retired rules, and generated AI content must not become agricultural authority.

5. Proposed AI output schema

```
{
  "schemaVersion": "1.0",
  "requestId": "req_example_001",
  "status": "COMPLETED",
  "candidates": [{
    "cropId": "crop-id-from-catalog",
    "suitability": "SUPPORTED",
    "reasons": [{
      "text": "Explanation tied to supplied data and approved knowledge.",
      "knowledgeIds": ["clxknowledge"],
      "dataFields": ["soil.nitrogen", "season.farmingMethod"]
    }],
    "alternatives": ["another-crop-id-from-catalog"],
    "warnings": [],
    "missingData": []
  }],
  "generalWarnings": [],
  "dataUsed": {
    "readingIds": ["clxreading"],
    "knowledgeVersions": [{ "knowledgeId": "clxknowledge", "version": 2 }]
  },
  "provider": "provider-name",
  "modelVersion": "model-version",
  "generatedAt": "2026-04-12T09:35:00.000Z"
}
```

Proposed suitability values: `SUPPORTED`, `REVIEW_NEEDED`, and `INSUFFICIENT_DATA`, aligned with current rule-engine concepts. Validate every response and cited knowledge ID server-side. Missing readings remain null/missing—not zero—and unsupported citations must be rejected.

6. Saving recommendations

CURRENT API `POST /api/recommendations` takes `{"seasonId":"..."}`. The backend checks session and farm permissions, resolves season context, requirements, the latest soil reading, and approved rules, runs the existing deterministic rules engine, then saves the recommendation.

Saved recommendations are linked to farmer, farm, plot, and season; versioned per season; and include an input snapshot, result JSON, provider/model metadata, status, timestamps, and references to the knowledge rules used.

Proposed AI flow:

1. Authenticated client asks CropCycle to assess a season.
2. CropCycle validates authorization and assembles the trusted input.
3. CropCycle's server calls the AI provider through an isolated adapter.
4. CropCycle validates the JSON, suitability values, and all citations.
5. CropCycle saves the input snapshot, output, provider/model version, and knowledge references.
6. CropCycle returns the saved recommendation. The AI provider does not write directly to CropCycle's database.

7. Offline and AI-unavailable behavior

If an AI assessment is required but its service is unreachable, return an explicit failure and do not save a recommendation:

```
HTTP 503 Service Unavailable
{
  "error": {
    "code": "AI_UNAVAILABLE",
    "message": "The recommendation service is currently unavailable. No AI recommendation was created.",
    "retryable": true
  }
}
```

Show the user that no AI recommendation was produced and offer a retry. Do not return an empty, success-shaped, or invented answer.

> **Decision required:** CropCycle already has a deterministic, approved-rule assessment engine. The teams must explicitly decide whether it may run as a clearly labeled fallback when AI is offline, or whether the AI request must return unavailable instead. There must be no silent substitution.

Separately, offline sensor readings can be queued in the browser and synchronized later. That reading queue is not an offline AI recommendation mechanism.

8. Input validation

Current online and batch-sync soil APIs apply these checks:

Measurement	Current accepted range and assumed unit
Nitrogen	0–1,000,000 ppm
Phosphorus	0–1,000,000 ppm
Potassium	0–1,000,000 ppm
Soil temperature	−50–100 °C
Humidity	0–100%

The APIs require finite numeric values and valid timestamps. They verify the selected plot is accessible, the season belongs to that plot, and the user has soil-recording permission. The units for N/P/K are current assumptions and should be confirmed against the sensor specification.

Older database readings may have null values. AI input must represent missing fields explicitly as null or omitted with a missing-data flag; never coerce them to zero. Before calling AI, verify the farmer/farm/plot/season relationships, timestamp, and authorization.

9. Samples and staging

NOT SET UP YET There is no dedicated staging deployment or synthetic AI integration seed-data package agreed at this point.

Proposed staging: a Vercel Preview deployment connected to a separate Neon branch or database—not production. Seed only synthetic records: a test farmer, farm, plot, seasons, readings, approved example rules, and past recommendation/outcome records. Exercise valid, missing-data, unauthorized, malformed-output, timeout, and provider-error cases without exposing real farmer data.

10. API, authentication, errors, and call sequence

Current CropCycle API authentication uses an HTTP-only session cookie named `cropcycle_session`. The browser calls CropCycle APIs; no direct AI-service authentication contract has been implemented.

> **Proposed security boundary:** browser → CropCycle backend → AI service. The backend holds the provider credential in a server-only environment variable; never expose it to browser code. Agree separately on service authentication, timeout, retry policy, and rate limits.

Purpose	Current route
Sign up	`POST /api/auth/signup`
Log in	`POST /api/auth/login`
Load authorized workspace	`GET /api/workspace`
List accessible saved recommendations	`GET /api/recommendations`
Assess and save using the current rules engine	`POST /api/recommendations` with `seasonId`
Read knowledge (role-filtered)	`GET /api/knowledge`
Record a soil reading	`POST /api/soil-readings`

HTTP status	Meaning
`400`	Malformed or invalid input
`401`	Missing or invalid session
`403`	Authenticated, but not permitted
`404`	Resource not found or not accessible
`409`	Duplicate, data conflict, or concurrent update
`500`	Unexpected server or database failure
`503`	Proposed response when the AI provider is unavailable

Decisions to confirm before implementation

1. Crop catalog: What canonical crop catalog will Crop Scientists maintain, and who owns it?

2. Soil history: Should AI receive only the latest reading, or a bounded series of timestamped readings?

3. Knowledge delivery: Include approved rule bodies in each request, or have the backend provide rules through a separate authenticated interface?

4. AI outage behavior: Is the approved-rule engine an allowed, explicitly labeled fallback, or must the AI endpoint return unavailable?

5. AI service contract: Which endpoint, authentication method, timeout, retry policy, and output schema version are expected?

6. History scope: Should history retrieval be farm-wide, plot-specific, or support both?

Current baseline

Until these decisions and proposed interfaces are implemented, the supported recommendation request is the current authenticated, permission-checked rule-based `POST /api/recommendations` with a `seasonId`. The expanded AI input/output schemas and farm-history endpoint in this document are proposals—not existing production endpoints.